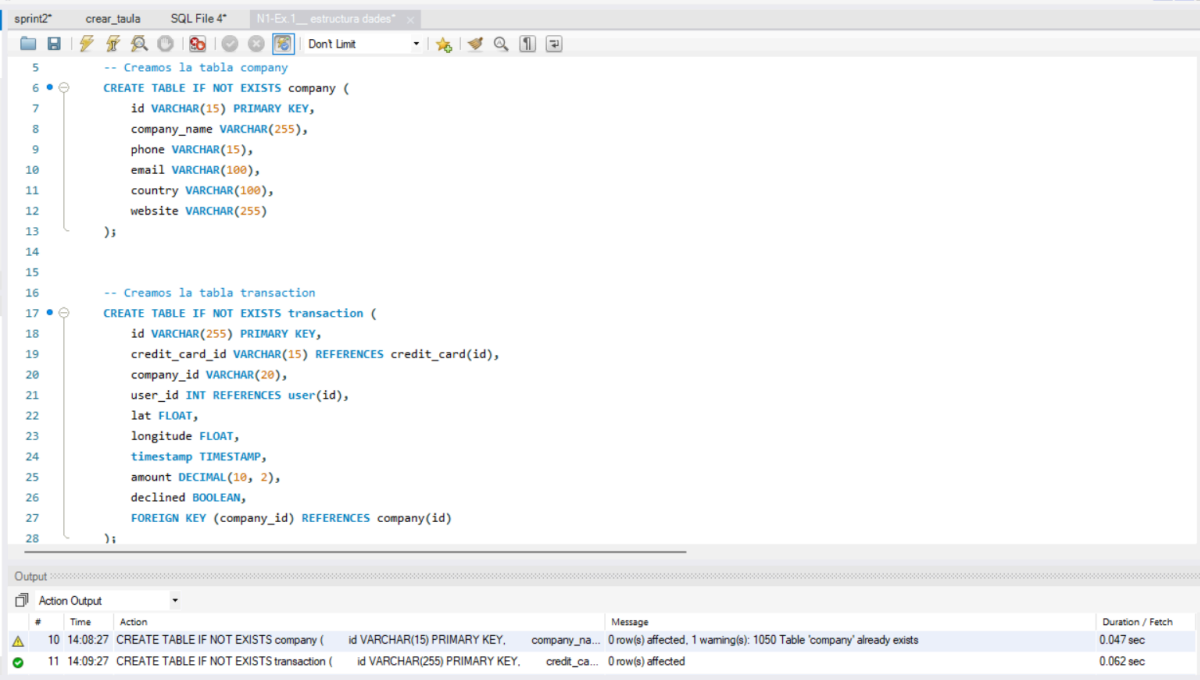


P2P - realitzat amb Alejandro Nicolás Pereyra.

Exercici 1

A partir dels documents adjunts (*estructura_dades* i *dades_introduir*), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Després d'executar l'script *estructura_dades* a Workbench, s'han creat dues taules:

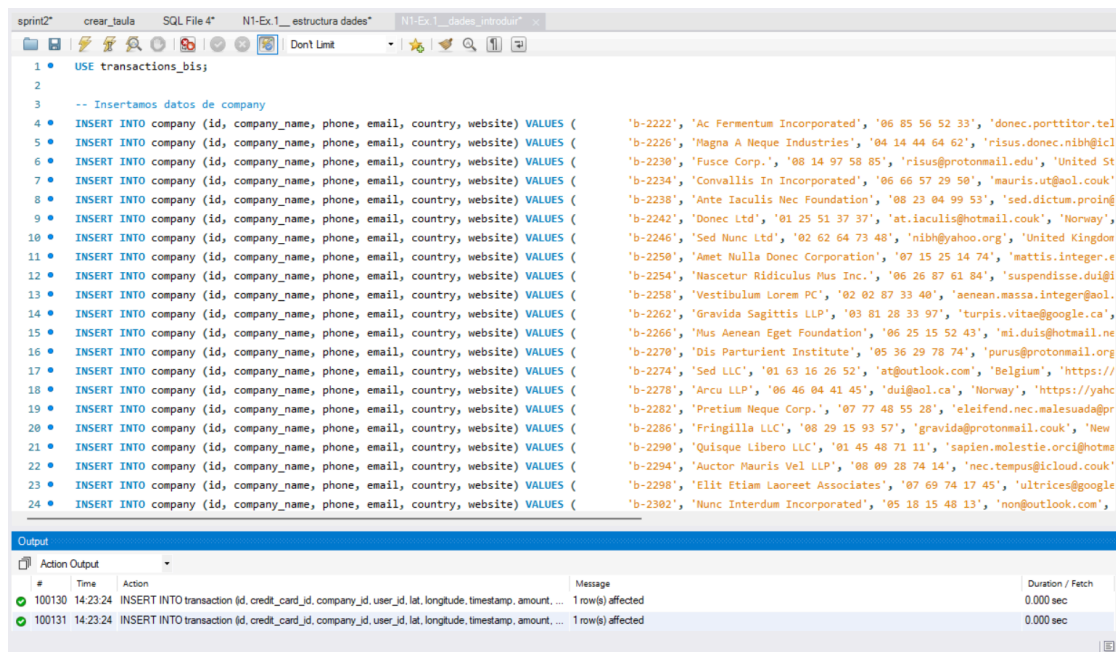


```
5 -- Creamos la tabla company
6 CREATE TABLE IF NOT EXISTS company (
7     id VARCHAR(15) PRIMARY KEY,
8     company_name VARCHAR(255),
9     phone VARCHAR(15),
10    email VARCHAR(100),
11    country VARCHAR(100),
12    website VARCHAR(255)
13 );
14
15
16 -- Creamos la tabla transaction
17 CREATE TABLE IF NOT EXISTS transaction (
18     id VARCHAR(255) PRIMARY KEY,
19     credit_card_id VARCHAR(15) REFERENCES credit_card(id),
20     company_id VARCHAR(20),
21     user_id INT REFERENCES user(id),
22     lat FLOAT,
23     longitude FLOAT,
24     timestamp TIMESTAMP,
25     amount DECIMAL(10, 2),
26     declined BOOLEAN,
27     FOREIGN KEY (company_id) REFERENCES company(id)
28 );
```

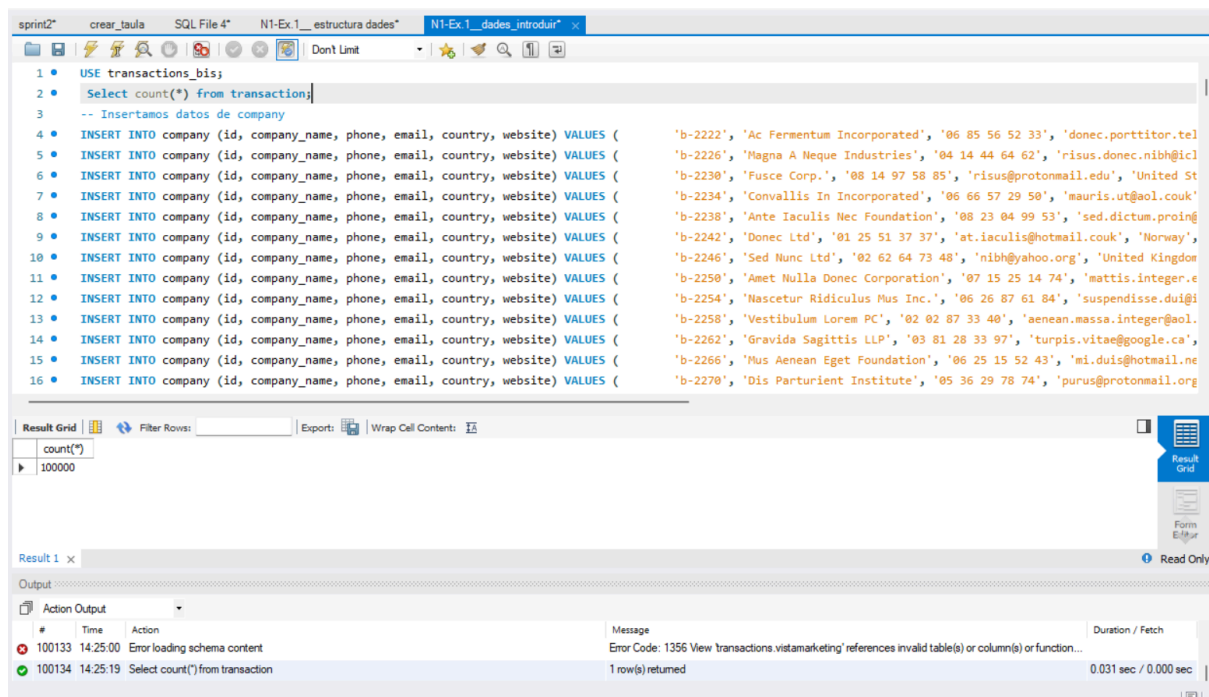
Output

#	Time	Action	Message	Duration / Fetch
10	14:08:27	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_na...	0 row(s) affected, 1 warning(s): 1050 Table 'company' already exists	0.047 sec
11	14:09:27	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_ca...	0 row(s) affected	0.062 sec

A continuació omplim les taules creades amb les corresponents dades:



I comprovem amb un count els 100000 registres nous a la taula:



l les 100 files de company:

```

1 • USE transactions_bis;
2 • Select count(*) from company;
3
4 -- Insertamos datos de company
5 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
6 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
7 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
8 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
9 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
10 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
11 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
12 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
13 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
14 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
15 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
16 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (

```

Result Grid

count(*)
100

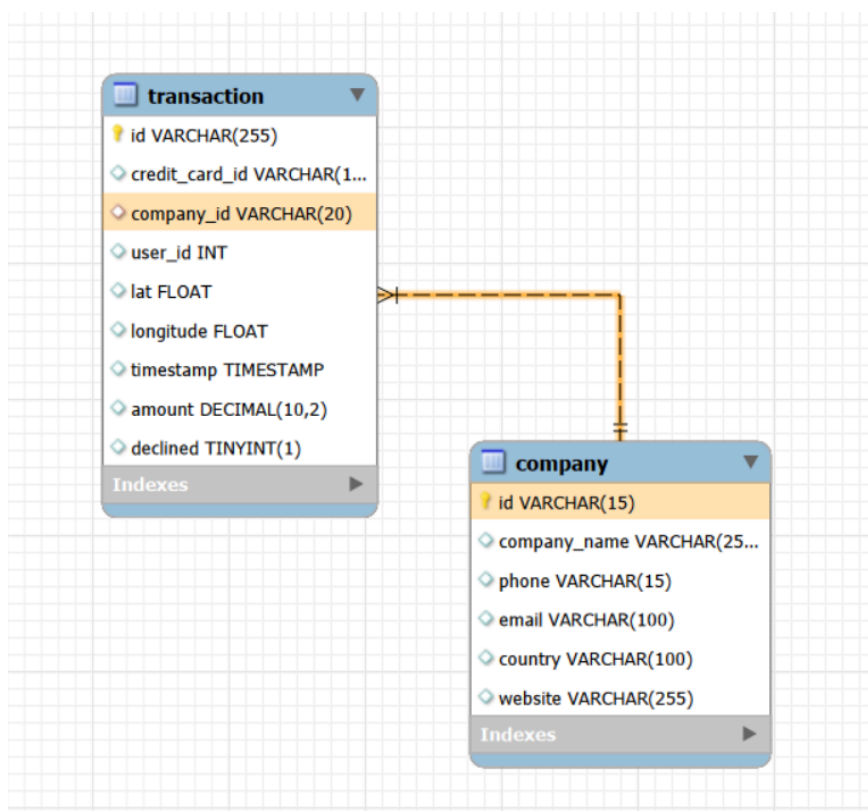
Result 2 x

Output

#	Time	Action	Message	Duration / Fetch
100137	14:32:23	SELECT company_id, id FROM transaction AS t WHERE EXISTS -- és dins del conjunt de company...	100000 row(s) returned	0.016 sec / 0.078 sec
100138	14:42:00	Select count(*) from company	1 row(s) returned	0.031 sec / 0.000 sec

Aquí representades en el DER (diagrama entitat-relació).

La taula *transaction* té com a clau primària la id que està marcada amb una clau groga i té com a FOREIGN_KEY company_id, que relaciona amb el camp id de la taula *company*.



Aquesta relació està ja a l'script que ens dona l'enunciat quan es crea la taula transaction:

```

14
15
16      -- Creamos la tabla transaction
17      CREATE TABLE IF NOT EXISTS transaction (
18          id VARCHAR(255) PRIMARY KEY,
19          credit_card_id VARCHAR(15) REFERENCES credit_card(id),
20          company_id VARCHAR(20),
21          user_id INT REFERENCES user(id),
22          lat FLOAT,
23          longitude FLOAT,
24          timestamp TIMESTAMP,
25          amount DECIMAL(10, 2),
26          declined BOOLEAN,
27          FOREIGN KEY (company_id) REFERENCES company(id)
28      );

```

La relació entre *company* i *transaction* és de 1 a N. És a dir, una companyia pot tenir més d'una transaction tal i com indica la pota de gall.

La taula *company* té tots els camps en format text (VARCHAR), tal com es mostra en el següent gràfic:

Table Name: company		Schema: transac								
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	VARCHAR(15)	☒	☒	☐	☐	☐	☐	☐	☐	
company_name	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
phone	VARCHAR(15)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
email	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
country	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
website	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

La taula *transaction* mostra major diversitat de format de les dades:

Table Name: transaction		Schema								
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	VARCHAR(255)	☒	☒	☐	☐	☐	☐	☐	☐	
credit_card_id	VARCHAR(15)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
company_id	VARCHAR(20)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
user_id	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
lat	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
longitude	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
timestamp	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	NULL
amount	DECIMAL(10,2)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
declined	TINYINT(1)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

En aquest cas cal assenyalar els camps *lat* i *longitude* que son amb FLOAT, donat el format de les coordenades geogràfiques, timestamp que és un camp de data/hora, amount és

decimal, en aquest cas pot tenir 10 dígit i fins a 2 decimals i per últim declined que és TINYINT és un integer de màxim un byte de memoria. En aquest cas està definit com a un sol dígit. Segurament funciona com un booleà o per marcar algún estat definit internament per un valor.

Prenc aquesta informació per construir les successives taules d'aquest SPRINT.

Exercici 2

Utilitzant **JOIN** realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```

1 • USE transactions;
2
3 • SELECT country, SUM(amount)
4 FROM company
5 INNER JOIN transaction
6 ON company.id = transaction.company_id
7 GROUP BY country;
8
9
10

```

The results pane displays a table with two columns: **country** and **SUM(amount)**. The data is as follows:

country	SUM(amount)
Germany	3466842.93
Australia	679418.68
United States	1069450.71
New Zealand	685567.09
Norway	785907.27
United Kingdom	3537120.34
Italy	3537120.34

The output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
10	11:06:50	SELECT country, amount FROM company INNER JOIN transaction ON company.id = transaction.co...	100000 row(s) returned	0.000 sec / 0.422 sec
11	11:08:05	SELECT country, SUM(amount) FROM company INNER JOIN transaction ON company.id = transacti...	Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains no...	0.016 sec
12	11:08:16	SELECT country, amount FROM company INNER JOIN transaction ON company.id = transaction.co...	100000 row(s) returned	0.000 sec / 0.359 sec
13	11:09:23	SELECT country, SUM(amount) FROM company INNER JOIN transaction ON company.id = transacti...	15 row(s) returned	0.407 sec / 0.000 sec

Hem fet un inner join de *company* i *transaction* a partir de les claus *id* i *company_id*. Agrupem els amounts per països fent un *GROUP BY* i ja tenim el llistat de països i les vendes.

- Des de quants països es generen les vendes.

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```

10 • SELECT COUNT(DISTINCT country) AS Total_paisos
11 FROM company
12 INNER JOIN transaction
13 ON company.id = transaction.company_id;

```

The results pane displays a table with one column: **Total_paisos**. The data is as follows:

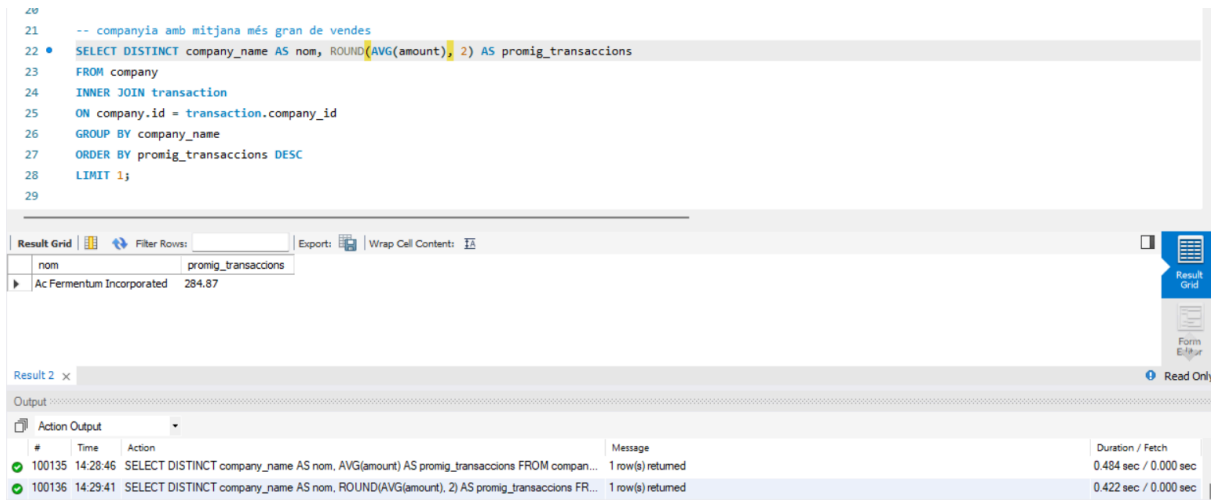
Total_paisos
15

The output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
17	11:20:42	SELECT country FROM company INNER JOIN transaction ON company.id = transaction.company_id...	15 row(s) returned	0.109 sec / 0.000 sec
18	11:22:19	SELECT COUNT(DISTINCT country) AS Total_paisos FROM company INNER JOIN transaction ON ...	1 row(s) returned	0.093 sec / 0.000 sec

Per saber el número de països, fem una relació de noms únics per mitjà de *distinct* amb un *count* per a obtenir el total de països. L'*output* és una sola línia amb el número.

- Identifica la companyia amb la mitjana més gran de vendes.



```
20
21 -- companyia amb mitjana més gran de vendes
22 • SELECT DISTINCT company_name AS nom, ROUND(AVG(amount), 2) AS promig_transaccions
23 FROM company
24 INNER JOIN transaction
25 ON company.id = transaction.company_id
26 GROUP BY company_name
27 ORDER BY promig_transaccions DESC
28 LIMIT 1;
29
```

nom	promig_transaccions
Ac Fermentum Incorporated	284.87

Result 2 x

Output

#	Time	Action	Message	Duration / Fetch
100135	14:28:46	SELECT DISTINCT company_name AS nom, AVG(amount) AS promig_transaccions FROM compan...	1 row(s) returned	0.484 sec / 0.000 sec
100136	14:29:41	SELECT DISTINCT company_name AS nom, ROUND(AVG(amount), 2) AS promig_transaccions FR...	1 row(s) returned	0.422 sec / 0.000 sec

Seguim amb el join de *company* i *transaction* i fem la mitja d'amount agrupades per empreses, les ordenem en ordre descendent i limitem la sortida a 1 empresa. Hem fet un DISTINCT per a les empreses i hem arrodonit a 2 dígit el promig. Aquesta és la que té el màxim de mitjana de vendes respecte a la resta.

Exercici 3

Utilitzant només subconsultes (sense utilitzar **JOIN**):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

27  -- ***** APARTAT 2 *****
28  -- transaccions realitzades per empreses d'Alemanya
29  • SELECT company_id, id
30    FROM transaction AS t
31   WHERE company_id IN -- és dins del conjunt de companyies alemanyes
32     (SELECT id FROM company
33      WHERE country = 'Germany');
34

```

The screenshot shows a SQL IDE interface. The top pane displays the SQL query. The middle pane shows the 'Result Grid' with two columns: 'company_id' and 'id'. It lists several transaction records for German companies. The bottom pane shows the 'Output' window with 'Action Output' selected, displaying the execution logs for two queries. The first query (SELECT COUNT) returned 1 row in 0.015 seconds. The second query (SELECT company_id, id) returned 13291 rows in 0.015 seconds.

company_id	id
b-2222	0F35A6C3-EDB0-41BE-808F-407F91FC1EA3
b-2222	0F4AF8B8-3E70-46F4-B8A9-DDF1D810ACA1
b-2222	0F646E8C-1A89-4505-92CC-A7FC82A401FF
b-2222	0FA47D77-7772-40D3-951D-9EBD5BA2AA80
b-2222	0FC032DD-01E5-4C20-88C0-650F2F52FE4E
b-2222	1001CA60-1259-4448-8AAB-3925D55C303E
b-2222	101EB281-4C45-4F7A-80DA-20567347E621
h-2222	10781436-70BF-476D-A719-4A8A9F48D1A6

#	Time	Action	Message	Duration / Fetch
64	12:40:28	SELECT COUNT(company_id) FROM transaction AS t WHERE company_id IN -- és dins del conjunt ...	1 row(s) returned	0.015 sec / 0.000 sec
65	12:42:12	SELECT company_id, id FROM transaction AS t WHERE company_id IN -- és dins del conjunt de co ...	13291 row(s) returned	0.000 sec / 0.015 sec

Fem una consulta sobre transaction en la que (where fa de filtre) el país sigui Alemanya. En el where afegim la subconsulta definint el país tal com demana l'enunciat.

Es podria usar EXISTS també, però segons bibliografia la diferencia de l'impacte sobre el rendiment no seria molt gran. No només depen del codi, també del motor de la base de dades i de la configuració de la taula.

L'output és la llista de transaccions. Hi he afegit la id de la companyia. Si volguessim el nom, caldria una altra subconsulta si no podem fer INNER JOIN.

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```

35 -- Empreses amb transaccions per un amount superior a la mitjana de totes les transaccions.
36 • SELECT id, company_name -- para ver el amout habria que hacer una consulta correlacionada
37 FROM company
38 WHERE id IN
39     (SELECT company_id
40      FROM transaction
41      WHERE amount >
42          (SELECT AVG(amount)
43           FROM transaction)
44     );

```

Result Grid

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.
b-2234	Convallis In Incorporated
b-2238	Ante Jaculis Nec Foundation
b-2242	Donec Ltd
b-2246	Sed Nunc Ltd
h-2250	Amet Nulla Donec Comoration

company 46 x

Output

#	Time	Action	Message	Duration / Fetch
62	12:36:58	SELECT id, company_name, amount FROM company WHERE id IN (SELECT company_id FROM...	Error Code: 1054. Unknown column 'amount' in field list	0.000 sec
63	12:39:41	SELECT id, company_name --para ver el amout habria que hacer una consulta correlacionada FRO...	100 row(s) returned	0.125 sec / 0.000 sec

Fem la consulta sobre company per que busquem el seu nom, però hem de filtrar per amount. Per tant en el where hi afegim la subconsulta a la taula *transaction*. Les dues van enllaçades pel camp id i company_id, que son les PRIMARY KEYS.

Per cercar les que estan per sobre de la mitjana, definim el filtre al segon WHERE fent una segona subconsulta per trobar la mitjana d'amount (AVG(amount)).

- Llistat d'empreses que no tenen transaccions registrades,

```

46 -- Llistat empreses que no tenen transaccions registrades
47 • SELECT company_name AS nom_empresa
48 FROM company
49 WHERE id NOT IN
50     (SELECT DISTINCT company_id
51      FROM transaction);
52 -- busco entre todas las empresas con transacciones cuáles no estan en la tabla company
53 -- el resultado es que todas tienen registros.
54

```

Result Grid

nom_empresa

company 76 x

Output

#	Time	Action	Message	Duration / Fetch
95	13:04:57	SELECT company_name FROM company WHERE id NOT IN (SELECT DISTINCT company_id FR...	0 row(s) returned	0.062 sec / 0.000 sec
96	13:06:33	SELECT company_name AS nom_empresa FROM company WHERE id NOT IN (SELECT DISTINC...	0 row(s) returned	0.062 sec / 0.000 sec

Busco les empreses de la taula *company* que no estàn (no tenen cap registre) a transaction. La subconsulta llista totes les empreses de *company* i busco si hi ha alguna d'aquestes que NO estigui a transaccions. Em surt el camp buit. Això vol dir que totes tenen registres de transaccions.

Exercici 4

Dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada tarjeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Creem la taula *credit_card*

```

60  -- **** APARTAT 4 ****
61  CREATE TABLE credit_card (
62  id VARCHAR(15) PRIMARY KEY, -- para que sea igual que en transaction
63  iban VARCHAR(50),
64  pan VARCHAR(20),
65  pin VARCHAR(4),
66  cvv VARCHAR(3),
67  expiring_date TIMESTAMP -- me da problemas al hacer la fk!
68  );
69

```

Output:

#	Time	Action	Message	Duration / Fetch
100138	14:42:00	Select count(*) from company	1 row(s) returned	0.031 sec / 0.000 sec
100139	14:50:48	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, -- para que sea igual que en transa...	0 row(s) affected	0.047 sec

Per evitar problemes a l'hora d'enllaçar amb transaction, convino el format de timestamp a VARCHAR. M'he adonat que després no puc crear la Foreign Key.

```

59
60  -- **** APARTAT 4 ****
61  CREATE TABLE credit_card (
62  id VARCHAR(15) PRIMARY KEY, -- para que sea igual que en transaction
63  iban VARCHAR(50),
64  pan VARCHAR(20),
65  pin VARCHAR(4),
66  cvv VARCHAR(3),
67  expiring_date TIMESTAMP -- me da problemas al hacer la fk!
68  );
69
70  ALTER TABLE credit_card CHANGE expiring_date expiring_date VARCHAR(10); -- paso la fecha a carácter
71
72  SHOW COLUMNS FROM credit_card; -- pruebo que se ha hecho el cambio de variable en la fecha

```

Output:

#	Time	Action	Message	Duration / Fetch
100139	14:50:48	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, -- para que sea igual que en transa...	0 row(s) affected	0.047 sec
100140	14:51:47	ALTER TABLE credit_card CHANGE expiring_date expiring_date VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec

Ara, expiring date es un VARCHAR:

```

72  SHOW COLUMNS FROM credit_card; -- pruebo que se ha hecho el cambio de variable en la fecha

```

Result Grid:

Field	Type	Null	Key	Default	Extra
iban	varchar(50)	YES			
pan	varchar(20)	YES			
pin	varchar(4)	YES			
cvv	varchar(3)	YES			
expiring_date	varchar(10)	YES			

Result 4 x

Output:

#	Time	Action	Message	Duration / Fetch
100140	14:51:47	ALTER TABLE credit_card CHANGE expiring_date expiring_date VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
100141	14:53:30	SHOW COLUMNS FROM credit_card	6 row(s) returned	0.000 sec / 0.000 sec

Carrego les dades a la nova taula:

```

69
70 • ALTER TABLE credit_card CHANGE expiring_date expiring_date VARCHAR(10); -- paso la fecha a carácter
71 • SHOW COLUMNS FROM credit_card; -- pruebo que se ha hecho el cambio de variable en la fecha
72
73 -- a continuación cargo los datos desde datos_introducir_credito.csv
74
75 • SELECT count(*) FROM credit_card;
76

```

Result Grid

count(*)
5000

Result 9 x

Output

#	Time	Action	Message	Duration / Fetch
110170	15:53:11	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9580', 'XX7812588...	1 row(s) affected	0.000 sec
110171	15:53:11	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9581', 'XX9156705...	1 row(s) affected	0.000 sec
110172	15:54:17	SELECT count(*) FROM credit_card	1 row(s) returned	0.000 sec / 0.000 sec

Ara la taula credit_card té 5000 valors

A continuació creem la relació amb transaction a partir de les id's de credit_card i les de transaction:

```

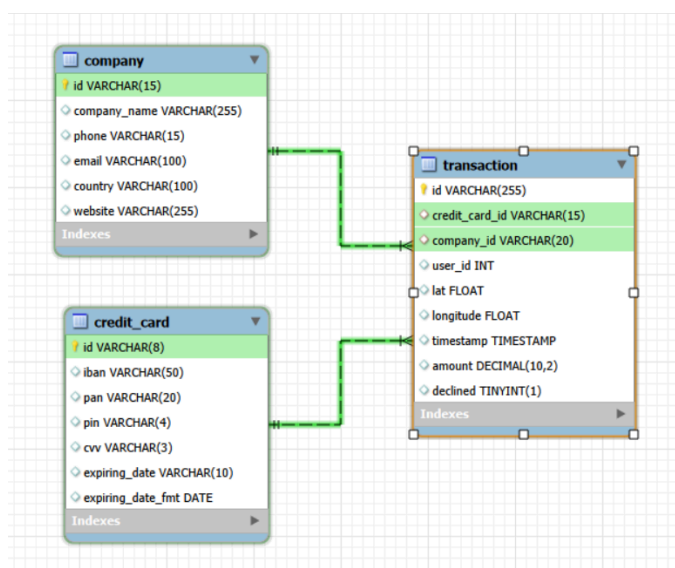
79 • ALTER TABLE transaction -- relaciono tablas
80 ADD CONSTRAINT fk_transaction_creditcard
81 FOREIGN KEY (credit_card_id) REFERENCES credit_card(id); -- me da error por registros NULL!!

```

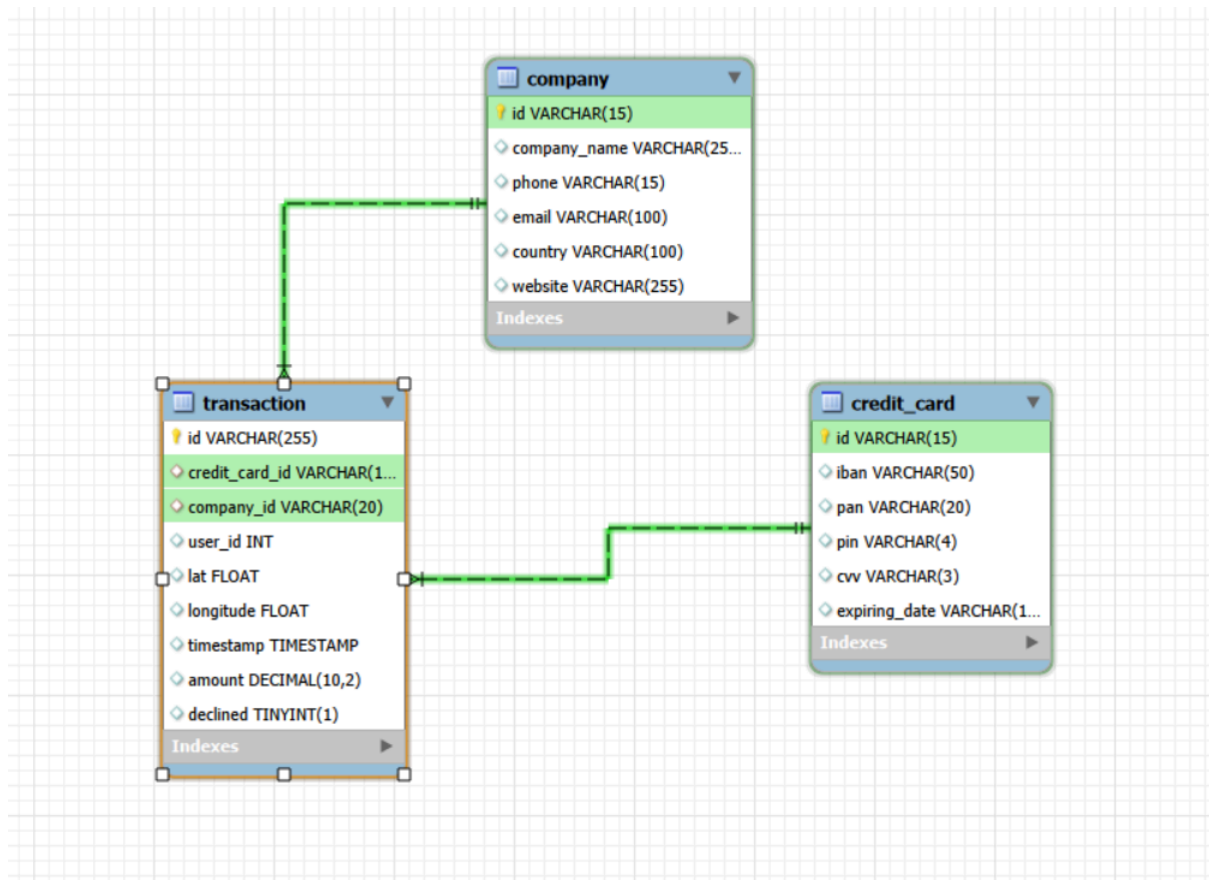
Output

#	Time	Action	Message	Duration / Fetch
110171	15:53:11	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9581', 'XX9156705...	1 row(s) affected	0.000 sec
110172	15:54:17	SELECT count(*) FROM credit_card	1 row(s) returned	0.000 sec / 0.000 sec
110173	15:55:54	ALTER TABLE transaction -- relaciono tablas ADD CONSTRAINT fk_transaction_creditcard FOREI...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.094 sec

El diagrama de relació de les taules queda així:



Així és com queda la relació de la tres taules. `credit_card` es relaciona amb `transaction` 1 és a molts (la pota de gall és a `transactions`). La *primary_key* de `credit_card` és la `id`, que es relaciona amb `transaction` a partir de `credit_card_id`:



Ara `transactions` té com a *primary_key* `id`, com ja tenia i dues *foreign_keys*, tal com es mostra a continuació:

transaction - Table	
Table Name: transaction	
Foreign Key Name	Referenced Table
fk_transaction_creditcard	`transactions`.`credit_card`
transaction_ibfk_1	`transactions`.`company`

Per a la manipulació de les dades he tingut problemes a la hora d'omplir la taula `credit_card` per incompatibilitat del format `DATE`.

La relació de `credit_card` amb `transactions` és de 1 - N.

Seguint alguna ajuda m'he adonat que com que hem definit a l'inici el camp `expiring_date` com a text, l'he passat a DATE fent:

```
SET expiring_date_fmt = STR_TO_DATE(expiring_date, '%m/%d/%y');
```

SET vol dir que escriu o defineix la columna `expiring_date_fmt` amb el format de data mes/dia/any. Si no, MySQL no em deixa avançar.

Per fer-ho hem creat una columna auxiliar per fer la conversió. Després, hem borrrat l'original i hem renombrat la nova columna:

```
83 • SET SQL_SAFE_UPDATES = 0; -- quito salvaguarda de datos
84 • UPDATE credit_card
85 • SET expiring_date_fmt = STR_TO_DATE(expiring_date, '%m/%d/%y'); -- modifco la cadena de texto a formato ISO
86 • SET SQL_SAFE_UPDATES = 1; -- vuelvo a activar la salvaguarda de datos
87 • ALTER TABLE credit_card DROP COLUMN expiring_date; -- elimino la columna de fecha en formato VARCHAR
88 • ALTER TABLE credit_card CHANGE expiring_date_fmt expiring_date DATE; -- renombro columna creada antes con fecha formato DATE
89 • describe credit_card;
90
```

Result Grid | Filter Rows: | Exports: | Wrap Cell Content: |

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	HULL	
iban	varchar(50)	YES		HULL	
pan	varchar(20)	YES		HULL	
pin	varchar(4)	YES		HULL	
cvv	varchar(3)	YES		HULL	
expiring_date	date	YES		HULL	

Result 10 x

Output

#	Time	Action	Message	Duration / Fetch
110179	16:03:22	ALTER TABLE credit_card DROP COLUMN expiring_date	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
110180	16:03:26	ALTER TABLE credit_card CHANGE expiring_date_fmt expiring_date DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
110181	16:03:52	describe credit_card	6 row(s) returned	0.031 sec / 0.000 sec

Ara `expiring_date` és del tipus DATE.

Per poder fer-ho hem hagut de treure les proteccions anti escriptura.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Primer comprovem que existeix el registre:

```
98 • SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'; -- està el registre?
99
```

id	iban
CcU-2938	TR301950312213576817638661

credit_card 1 x

Output

#	Time	Action	Message	Duration / Fetch
1	16:03:27	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.016 sec / 0.000 sec

I en efecte apareix a la taula.

Selecciono el id 'CcU-2938' i modifico l'IBAN pel de l'enunciat. La modificació es fa per mitja d'un **UPDATE** taula **SET** iban = 'TR323456312213576817699999' de lid esmentada.

```
100
101 • UPDATE credit_card SET iban = 'TR323456312213576817699999' -- actualizo el IBAN del registro
102 WHERE id = 'CcU-2938';
103
```

credit_card 1 x

Output

#	Time	Action	Message	Duration / Fetch
1	16:03:27	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.016 sec / 0.000 sec
2	16:07:07	UPDATE credit_card SET iban = 'TR323456312213576817699999' -- actualizo el IBAN del registro W...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.047 sec

Llenço la consulta al final para mostrar que s'ha fet el canvi correctament i veiem l'IBAN nou.

```
105 • SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'; -- verifiquem sobre l'id
```

id	iban
CcU-2938	TR323456312213576817699999

credit_card 2 x

Output

#	Time	Action	Message	Duration / Fetch
2	16:07:07	UPDATE credit_card SET iban = 'TR323456312213576817699999' -- actualizo el IBAN del registro ...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.047 sec
3	16:09:34	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 6

En la taula "transaction" ingressa una nova transacció:

Per afegir un registre a una taula he fet servir:

INSERT INTO taula(llista de columnes) VALUES (valors seguint ordre de la llista de columnes)

L'ordre correcte és primer afegir els registres pare a les taules *company* i *credit_card* i els fills a la taula *transaction*.

```
114 • INSERT INTO company (id) VALUES ('b-9999'); -- inserto registros padre
115 • INSERT INTO credit_card (id) VALUES ('CcU-9999');
116
```

#	Time	Action	Message	Duration / Fetch
9	16:17:00	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected	0.016 sec
10	16:17:48	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.000 sec

A continuació a transaction fem INSERT INTO dels valors del registre de l'enunciat:

```
113 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) -- adjunto los valores de registro hijo en transaction
114 • VALUES ('10881D1D-5823-A76C-55EF-C568E49A9900', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0); -- que referencia a transaction
115
116
```

#	Time	Action	Message	Duration / Fetch
10	16:17:48	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.000 sec
11	16:22:47	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) V...	1 row(s) affected	0.031 sec

S'ha de seguir aquest ordre per no crear línies orfes en les taules. A més, és l'ordre lògic per què no podem inserir una transacció si no hi ha la id de l'empresa ni la id de la tarjeta de crèdit.

Per finalitzar comprovo que el registre s'ha creat correctament:

```
118 • SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999'; -- busco el nou registre a partir de la credit_card_id
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
10881D1D-5823-A76C-55EF-C568E49A9900	CcU-9999	b-9999	9999	829.999	-117.999	111.11	0	0

#	Time	Action	Message	Duration / Fetch
14	16:30:48	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999'	1 row(s) returned	0.015 sec / 0.000 sec
15	16:30:54	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999'	1 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

Fem un describe per mostrar que hi ha la columna pan:

```

118  -- **** APARTAT 7 ****
119
120  • DESCRIBE credit_card;
121

```

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI		
iban	varchar(50)	YES			
pan	varchar(20)	YES			
pin	varchar(4)	YES			
cvv	varchar(3)	YES			
expiring_date	date	YES			

#	Time	Action	Message
110190	16:17:57	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected
110191	16:18:32	select * from transaction where credit_card_id = 'CcU-9999'	1 row(s) returned
110192	16:20:10	DESCRIBE credit_card	6 row(s) returned

Per eliminar la columna *pan* columna he fet servir **ALTER TABLE** taula **DROP** columna.

La sortida de la consulta es pot veure que no hi ha la columna esmentada, tal com demana l'enunciat.

```

110  -- **** APARTAT 7 ****
111
112  • ALTER TABLE credit_card DROP COLUMN Pan;
113  • SHOW COLUMNS FROM credit_card;
114

```

Field	Type	Null	Key	Default	Extra
id	varchar(8)	NO	PRI		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	varchar(3)	YES			
expiring_date	varchar(10)	YES			
expiring_date_fmt	date	YES			

#	Time	Action	Message	Duration / Fetch
24	11:19:30	SHOW COLUMNS FROM credit_card	7 row(s) returned	0.000 sec / 0.000 sec
25	11:19:44	ALTER TABLE credit_card DROP COLUMN Pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec
26	11:19:54	SHOW COLUMNS FROM credit_card	6 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 8

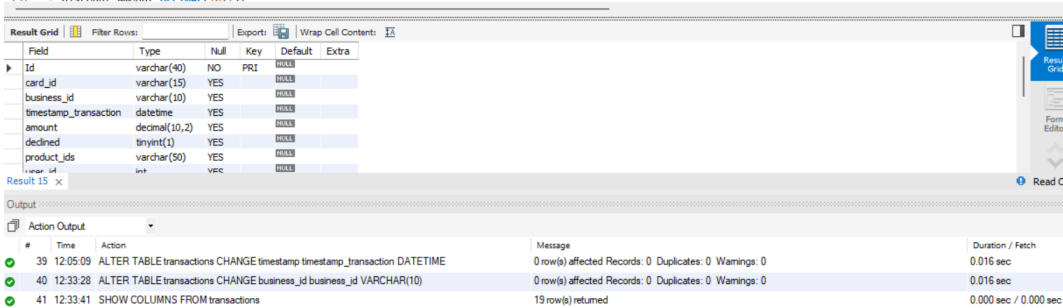
Dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules.

Abans de tot hem creat la base de dades i l'he seleccionat per al seu ús:

```
CREATE DATABASE nova_transaction; # aquí creo la base de dades
USE nova_transaction; # per fer servir aquesta base de dades d'ara endavant.
```

A continuació hi ha l'output de la creació de la taula transactions:

```
119
120 • CREATE TABLE transactions (
121   Id VARCHAR(40) PRIMARY KEY,
122   card_id VARCHAR(15),
123   business_id VARCHAR(5),
124   timestamp DATETIME,
125   amount DECIMAL(10,2),
126   declined TINYINT(1),
127   product_ids VARCHAR(50),
128   user_id INT,
129   lat DOUBLE,
130   longitude DOUBLE, -- usamos DOUBLE pq son coordenadas y necesitamos alta precisión
131   discount amount DECIMAL(10,2).
```



The screenshot shows a database management tool interface. The top part displays the SQL code for creating the 'transactions' table. Below that, a table structure view shows the following fields: Id (varchar(40), NO, PRI, NULL), card_id (varchar(15), YES, NULL), business_id (varchar(10), YES, NULL), timestamp_transaction (datetime, YES, NULL), amount (decimal(10,2), YES, NULL), declined (tinyint(1), YES, NULL), product_ids (varchar(50), YES, NULL), user_id (int, YES, NULL), lat (double, YES, NULL), longitude (double, YES, NULL), and discount (decimal(10,2), YES, NULL). The bottom part shows the output of SQL commands: 39 ALTER TABLE transactions CHANGE timestamp_transaction DATETIME (0 rows affected), 40 ALTER TABLE transactions CHANGE business_id VARCHAR(10) (0 rows affected), and 41 SHOW COLUMNS FROM transactions (19 rows returned).

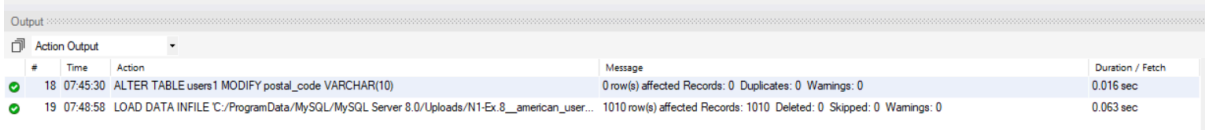
Primer he obert el CSV en excel per veure com són les dades. He copiat la primera fila que son els camps i a partir d'aquí he construït la taula definint cada camp segons tipus de dada i la seva longitud o mida.

La resta de taules les he creat de la mateixa manera, manualment amb codi.

Creo una taula *users1* pels usuaris americans. El camp data també funciona malament i el transformo a un VARCHAR. Afegeixo la columna amb el nom de continent per no perdre la granularitat de les dades. Ja que he vist que més endavant hem d'unir les columnes a una sola.

Creo la taula i importo les dades dels usuaris americans:

```
217 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv' -- atención con las barras invertidas!!!
218 INTO TABLE users1
219 FIELDS TERMINATED BY ','
220 OPTIONALLY ENCLOSED BY '"'
221 LINES TERMINATED BY '\n'
222 IGNORE 1 ROWS
223 (id,name,surname,phone,email,birth_date,country,city,postal_code,address,signup_date,user_segment,income_band);
224
```



The screenshot shows a database management tool interface. The top part displays the SQL code for loading data from a CSV file into the 'users1' table. Below that, the output shows the execution of the command: 18 ALTER TABLE users1 MODIFY postal_code VARCHAR(10) (0 rows affected) and 19 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_user...' (1010 row(s) affected, 1010 Deleted, 0 Skipped, 0 Warnings, 0.063 sec).

Hi apareixien 1010 usuaris americans.

A continuació edito el format de la columna birth_date:

```

226 • ALTER TABLE users1 ADD COLUMN birth_date_fmt DATE;
227 • SET SQL_SAFE_UPDATES = 0; -- quito protección de modificación
228 • UPDATE users1
229   SET birth_date_fmt = STR_TO_DATE(birth_date, '%b %e, %Y'); -- texto a fecha
230 • ALTER TABLE users1 DROP COLUMN birth_date;
231 • ALTER TABLE users1 CHANGE birth_date_fmt birth_date DATE;
232 • SET SQL_SAFE_UPDATES = 1; -- rehabilito protección de modificación
233

```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
20	07:50:30	ALTER TABLE users1 ADD COLUMN birth_date_fmt DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
21	07:50:43	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec

Ara a users1 el birthdate és anys mes i dia.

Creo la taula users2 amb les dades dels usuaris europeus. Aquesta vegada afegint la columna continent al moment de crear-la i amb el camp per defecte amb el valor 'european'.

The screenshot shows a database management tool interface. The top part displays the SQL code for creating a table named 'users2'. The code defines columns: id (INT PRIMARY KEY), name (VARCHAR(30)), surname (VARCHAR(30)), phone (VARCHAR(20)), email (VARCHAR(50)), birth_date (VARCHAR(20)), country (VARCHAR(30)), city (VARCHAR(30)), postal_code (VARCHAR(10)), address (VARCHAR(60)), signup_date (DATE), user_segment (VARCHAR(30)), income_band (VARCHAR(15)), and continent (VARCHAR(50) NOT NULL DEFAULT 'european').

Below the code, the 'Result Grid' shows the data for the 'users2' table. The data includes columns: id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, and user_segment. The data rows show users from the United Kingdom, including Meghan Hayden, Keegan Pugh, Cooper Bullock, Joshua Russell, Remedios Case, and Philin Carey.

The bottom part of the screenshot shows the 'Output' window with the following actions:

- 43 07:58:39 ALTER TABLE users1 ADD COLUMN continent VARCHAR(50) NOT NULL DEFAULT 'american' (0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0, 0.078 sec)
- 44 07:58:59 select * from users1 (1010 row(s) returned, 0.000 sec / 0.000 sec)
- 45 08:00:45 CREATE TABLE users2 (-- aquí importo europeos id INT PRIMARY KEY, name ... (0 row(s) affected, 0.063 sec)

I la completem amb les dades:

```

260 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_european_users.csv'
261 INTO TABLE users2
262 FIELDS TERMINATED BY ','
263 OPTIONALLY ENCLOSED BY '"'
264 LINES TERMINATED BY '\n'
265 IGNORE 1 ROWS
266 (id,name,surname,phone,email,birth_date,country,city,postal_code,address,signup_date,user_segment,income_band);
267

```

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band
151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	2013-07-08	young_professional	
152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street	2014-12-23	frequent_shopper	
153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consectetur St.	2014-10-07	young_professional	
154	Cooper	Bullock	(021) 2521 6627	et.154@outlook.net	Nov 2, 1986	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.	2013-07-01	occasional	
155	Joshua	Russell	055 4409 5286	justo.nec.ante@outlook.edu	Jan 23, 1984	United Kingdom	Manchester	M1 1AE	Ap #285-4727 Auctor. Av.	2013-03-05	frequent_shopper	
156	Remedios	Case	055 3114 1566	mollis.phasellus.libero@aol.com	Oct 9, 1994	United Kingdom	Liverpool	L1 1AA	479-3690 Turpis Road	2014-09-08	family	
157	Phlin	Carry	0800 640 6751	rhacellus@vahoo.net	Oct 10, 1992	United Kingdom	Manchester	M1 1AF	196-11013 Culcras Street	2014-12-22	frequent_shopper	

users2 13 x

Output

#	Time	Action	Message	Duration / Fetch
45	08:00:45	CREATE TABLE users2 (- aqui importo europeos id INT PRIMARY KEY, name ...	0 row(s) affected	0.063 sec
46	08:00:53	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_european_use...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.156 sec
47	08:00:58	select * from users2	3990 row(s) returned	0.000 sec / 0.015 sec

Amb 3990 valors.

En aquest cas també canviem el format de la data:

```

271 -- cambiar formato fecha
272 • ALTER TABLE users2 ADD COLUMN birth_date_fmt DATE;
273 • SET SQL_SAFE_UPDATES = 0; -- quito protección de modificación
274 • UPDATE users2
275 SET birth_date_fmt = STR_TO_DATE(birth_date, '%b %e, %Y'); -- texto a fecha
276 -- ALTER TABLE users2 DROP COLUMN birth_date;

```

Output

#	Time	Action	Message	Duration / Fetch
49	08:03:59	ALTER TABLE users2 ADD COLUMN birth_date_fmt DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
50	08:04:03	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
51	08:04:07	UPDATE users2 SET birth_date_fmt = STR_TO_DATE(birth_date, '%b %e, %Y')	3990 row(s) affected Rows matched: 3990 Changed: 3990 Warnings: 0	0.141 sec

I queda birth_date amb format DATE com volíem:

```

271 -- cambiar formato fecha
272 • ALTER TABLE users2 ADD COLUMN birth_date_fmt DATE;
273 • SET SQL_SAFE_UPDATES = 0; -- quito protección de modificación
274 • UPDATE users2
275 SET birth_date_fmt = STR_TO_DATE(birth_date, '%b %e, %Y'); -- texto a fecha
276 • ALTER TABLE users2 DROP COLUMN birth_date;
277 • ALTER TABLE users2 CHANGE birth_date_fmt birth_date DATE;
278
279 • SET SQL_SAFE_UPDATES = 1; -- rehabilito protección de modificación
280

```

Field	Type	Null	Key	Default	Extra
postal_code	varchar(10)	YES			INDEX
address	varchar(60)	YES			INDEX
signup_date	date	YES			
user_segment	varchar(30)	YES			
income_band	varchar(15)	YES			
continent	varchar(50)	NO		europ...	
birth_date	date	YES			INDEX

Result 17 x

Output

#	Time	Action	Message	Duration / Fetch
59	08:07:03	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
60	08:07:06	describe users2	14 row(s) returned	0.000 sec / 0.000 sec

Ara relacionem la taula amb transaction creant la foreign_key.

```

283
284 • ALTER TABLE transaction -- relaciono tablas
285 ADD CONSTRAINT fk_transaction_id_users
286 FOREIGN KEY (user_id) REFERENCES users1(id); -- ya no me da error!!

```

Output

#	Time	Action	Message	Duration / Fetch
60	08:07:06	describe users2	14 row(s) returned	0.000 sec / 0.000 sec
61	08:08:52	ALTER TABLE transaction -- relaciono tablas ADD CONSTRAINT fk_transaction_id_users FOREIGN...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec

Les taules usuaris europeus i usuaris USA s'han hagut de fusionar primer fent un **UNION ALL**, ja que transaction tenia usuaris amb codis USA i codis Europa:

```

293 • CREATE TABLE junts_users AS -- fusiono las tablas de usuarios, dado que en transactions hay datos de USA y de Europa. es una nueva dimension de la BBDD.
294 SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, continent
295 FROM users1
296 UNION ALL
297 SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, continent
298 FROM users2;

```

Output

#	Time	Action	Message	Duration / Fetch
61	08:08:52	ALTER TABLE transaction -- relaciono tablas ADD CONSTRAINT fk_transaction_id_users FOREIGN...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
62	08:10:58	CREATE TABLE junts_users AS -- fusiono las tablas de usuarios, dado que en transactions hay dato...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.203 sec

El resultat és una taula amb 5000 usuaris.

Quan es fa un UNION cal tenir en compte l'ordre de les columnes. Ha de ser el mateix a les dues taules. La columna final es 'continent', que ja hem dit que serveix per no perdre granularitat.

Creo a continuació la *primary key* sobre la nova taula fusionada i la *Foreign_key* sobre *transaction* amb *ALTER TABLE* i la relaciono amb *transaction*.

```

301 -- relaciono amb transaction
302 • ALTER TABLE transaction
303 ADD CONSTRAINT fk_transaction_users_junts
304 FOREIGN KEY (user_id) REFERENCES junts_users(id);
305

```

Output

#	Time	Action	Message	Duration / Fetch
92	08:23:58	ALTER TABLE transaction -- relaciono tablas ADD CONSTRAINT fk_transaction_id_users FOREIGN...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('nova_transaction'...	0.078 sec
93	08:24:25	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_users_junts FOREIGN KEY (user_id) ...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	1.922 sec

Creem la taula *credit_card* (estava creada reaprofitant codi anterior però no estava explicat en l'escript) i hi carreguem les dades:

```

311 CREATE TABLE credit_card (
312     id VARCHAR(20) PRIMARY KEY,
313     user_id INT,
314     iban VARCHAR(50),
315     pan VARCHAR(50),
316     pin VARCHAR(4),
317     cvv VARCHAR(4),
318     track1 VARCHAR(255),
319     track2 VARCHAR(255),
320     expiring_date DATE,
321     card_type VARCHAR(30),
322     card_renewal_flag TINYINT(1)
323 );
324
325 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
326 INTO TABLE credit_card
327 FIELDS TERMINATED BY ',' ENCLOSED BY '"'
328 LINES TERMINATED BY '\n'
329 IGNORE 1 LINES
330 (id, user_id, iban, pan, pin, cvv, track1, track2, @expiring_date, card_type, card_renewal_flag)
331 SET expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y');

```

Output:

#	Time	Action	Message	Duration / Fetch
29	17:29:57	CREATE TABLE credit_card (id VARCHAR(20) PRIMARY KEY, user_id INT, iban VARCHAR...	0 row(s) affected. 1 warning(s): 1681 Integer display width is deprecated and will be removed in a futur...	0.063 sec
30	17:30:08	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.c...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.172 sec

Aquesta vegada, per no fer el canvi de format com abans, farem servir una variable d'usuari definida amb '@'. Quan carreguem les dades des del CSV, aquestes s'importen per ordre tal com hem definit en el parèntesis de les columnes en el LOAD DATA. El camp @expiring_date no es carrega a la taula, es guarda en memòria momentàniament per després definir la columna de la taula final amb el SET expiring_date i l'omplim amb la variable que hem definit fent l'STR_TO_DATE. Ens estalviem de fer servir una columna intermitja.

Comprovem que hem creat la taula està completa:

```

333 DESCRIBE credit_card;
334 SELECT * FROM credit_card;

```

Result Grid:

	id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date	card_type	c
CS-4857	276	ES415679416676968711699208	2314242385113924	1819	467	%82314242385113924~LWCBUDLWCBUD~22...	%82314242385113924=24101015183631647	2028-09-25	debit	1	
CS-4858	277	NL252053037609851262242244	6582720299715533	3964	817	%86582720299715533~TQMVITITQMV~2404...	%86582720299715533=24111011045462722	2028-12-28	debit	0	
CS-4859	278	SE720084318584364771497262	8861684536289642	4983	277	%88861684536289642~COFBDGCOFBDG~280...	%88861684536289642=2502101761665371?	2028-11-26	credit	0	
CS-4860	279	ES165574078045314365283427	2481155515498459	6876	661	%82481155515498459~TLUTJTUTLUTJ~31040...	%82481155515498459=2602101514414395?	2028-07-27	prepaid	0	
CS-4861	280	SE556762210431025196620841	1308930301149557	5710	398	%81308930301149557~HPOBNZ~HPOBNZ~330...	%81308930301149557=2805101751305028?	2028-04-26	credit	0	
CS-4862	281	NL949396429710439685071325	6715617009807829	4042	174	%86715617009807829~LDMVTD~LDMVTD~33...	%86715617009807829=2210101702370428?	2028-11-26	credit	0	
CS-4863	282	FR628211851201116231488674	3140879819451394	5969	449	%83140879819451394~OXXJOD~OXXJOD~230...	%83140879819451394=3210101158648599?	2028-12-29	debit	0	

credit_card 7 x

Output:

#	Time	Action	Message	Duration / Fetch
32	17:37:06	DESCRIBE credit_card	11 row(s) returned	0.031 sec / 0.000 sec
33	17:37:23	SELECT * FROM credit_card	5000 row(s) returned	0.000 sec / 0.016 sec

Ara la relacionem amb TRANSACTION. En aquest cas una targeta pot tenir moltes transactions. La relació serà 1 - N.

```

337 -- relaciono amb transaction
338 ALTER TABLE transaction
339 ADD CONSTRAINT fk_transaction_credit_cards
340 FOREIGN KEY (card_id) REFERENCES credit_card(id); -- FK = la PK de credit card apunta a card_id de transaction

```

Output:

#	Time	Action	Message	Duration / Fetch
34	17:45:17	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_cards FOREIGN KEY (user_id)...	Error Code: 3780. Referencing column 'user_id' and referenced column 'id' in foreign key constraint fk...	0.016 sec
35	17:46:26	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_cards FOREIGN KEY (card_id)...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.656 sec

Creem la taula companies i pujem les dades:

```

357     email VARCHAR(100),
358     country VARCHAR(50),
359     website VARCHAR(255),
360     merchant_category VARCHAR(50),
361     merchant_price_position VARCHAR(20)
362 );
363 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_companies.csv'
364 INTO TABLE companies
365 FIELDS TERMINATED BY ','
366 LINES TERMINATED BY '\n'
367 IGNORE 1 LINES
368 (company_id, company_name, phone, email, country, website,
369 merchant_category, merchant_price_position);
370
371 • SELECT COUNT(*) FROM companies;
  
```

Result Grid

COUNT(*)
100

Result 11 x

Output

#	Time	Action	Message	Duration / Fetch
43	11:28:22	CREATE TABLE companies (company_id VARCHAR(15) PRIMARY KEY, company_name VAR...	0 row(s) affected	0.063 sec
44	11:28:59	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_companies.cs...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.015 sec

I comprovem que estigui completa amb un count. Obtenim 100 registres.

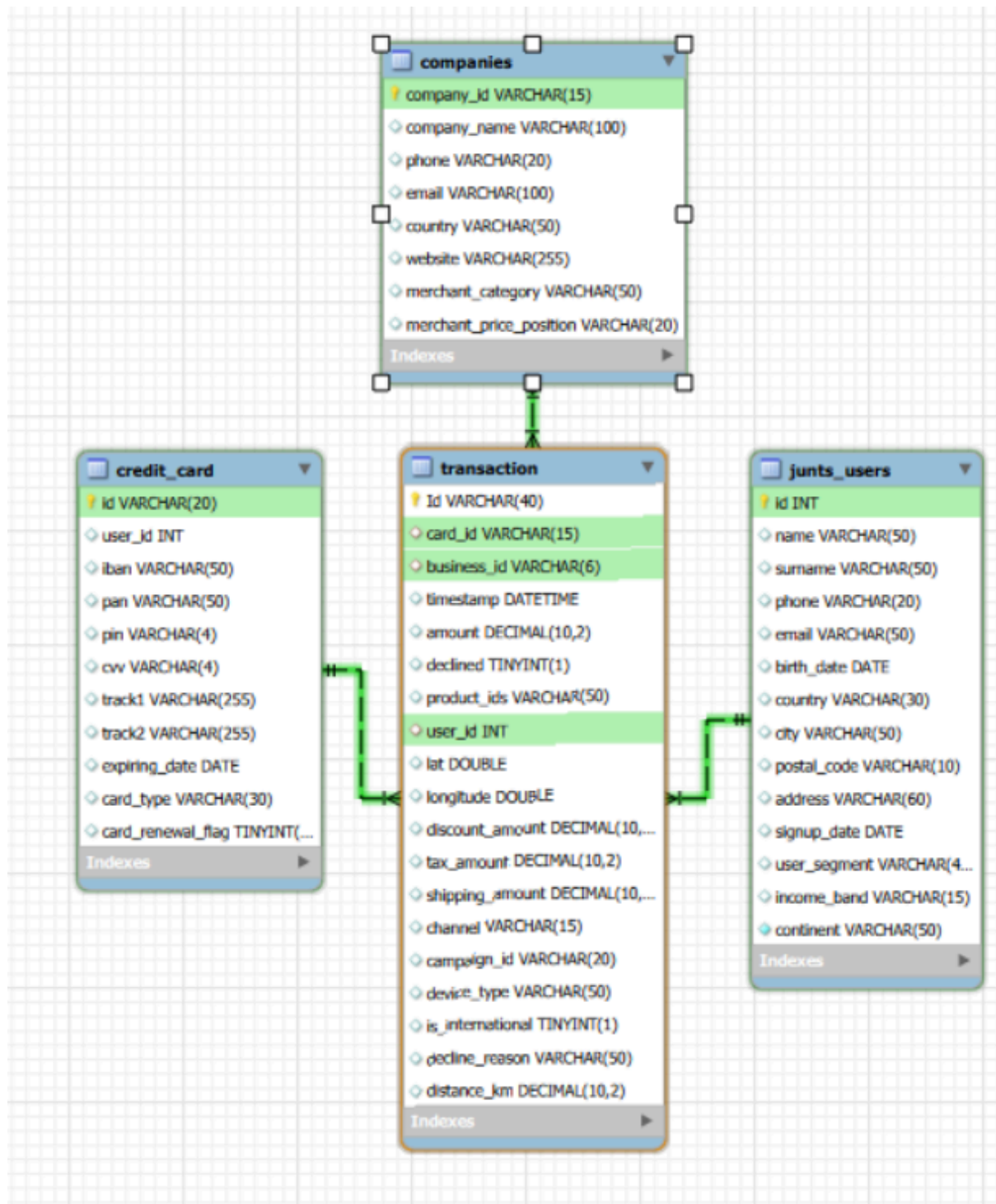
```

375 -- añadimos las foreign keys con una relacion de 1-N con transaction (una compañía puede tener varias transaction)
376 • ALTER TABLE transaction
377 ADD CONSTRAINT fk_transaction_companies
378 FOREIGN KEY (business_id) REFERENCES companies(company_id);
379
  
```

Output

#	Time	Action	Message	Duration / Fetch
2	18:57:11	SELECT COUNT(*) FROM transaction LEFT JOIN companies ON transaction.business_id = compa...	1 row(s) returned	0.328 sec / 0.000 sec
3	18:57:46	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_companies FOREIGN KEY (business...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.797 sec

La relació de les taules en la base de dades *transactions* de l'esquema *nova_transaction* queda així:



Amb les corresponents *FOREIGN KEYS* a *transactions* per crear la relació de les taules:

transaction - Table ×

 Table Name:

Foreign Key Name	Referenced Table
fk_transaction_companies	`nova_transaction`.`companies`
fk_transaction_credit_car...	`nova_transaction`.`credit_card`
fk_transaction_users_junts	`nova_transaction`.`junts_users`

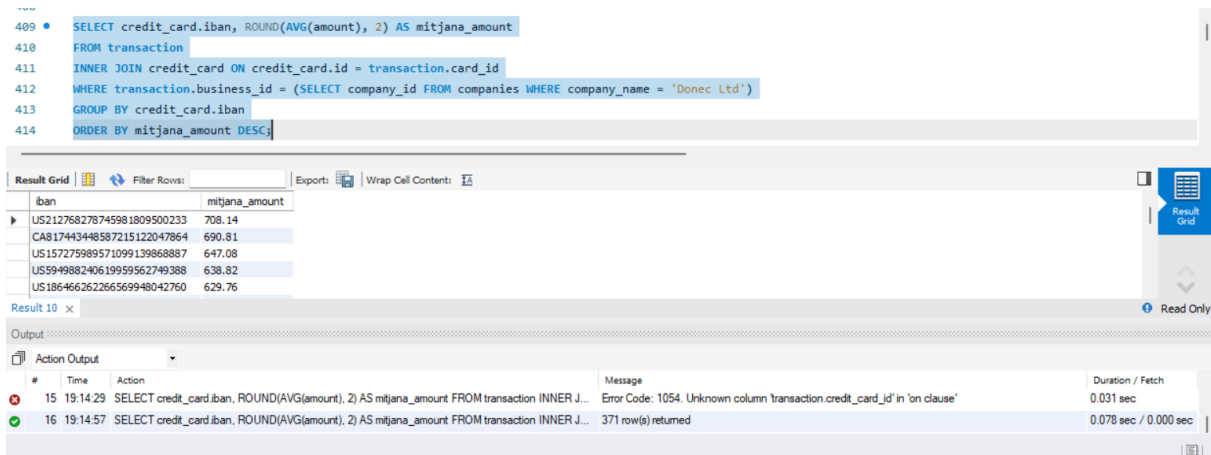
Faig servir la taula de usuaris consolidats (*junts_users*) i la de *transaction*. Busco id i nom dels usuaris i en el filtre del *where* hi poso la subconsulta que apunta a *transaction* enllaçada amb les id's. Aquesta subconsulta és per trobar les id's que es repeteixen més de 80 vegades a les transaccions i els compta. A la sortida tenim 4 usuaris.

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Faig un INNER JOIN entre *transaction* i *credit_card* primer i al filtre del nom de les companyies hi adjunto una subconsulta per filtrar en les empreses, la de Donec. Hem arrodonit a 2 decimals i he ordenat de més a menys les mitjanes.

```
409 • SELECT credit_card.iban, ROUND(AVG(amount), 2) AS mitjana_amount
410 FROM transaction
411 INNER JOIN credit_card ON credit_card.id = transaction.card_id
412 WHERE transaction.business_id = (SELECT company_id FROM companies WHERE company_name = 'Donec Ltd')
413 GROUP BY credit_card.iban
414 ORDER BY mitjana_amount DESC;
```



iban	mitjana_amount
US212768278745981809500233	708.14
CA817443448587215122047864	690.81
US157275989571099139868887	647.08
US594988240619959562749388	638.82
US186466262266569948042760	629.76

Result 10 x

Output

#	Time	Action	Message	Duration / Fetch
15	19:14:29	SELECT credit_card.iban, ROUND(AVG(amount), 2) AS mitjana_amount FROM transaction INNER J...	Error Code: 1054. Unknown column 'transaction.credit_card_id' in 'on clause'	0.031 sec
16	19:14:57	SELECT credit_card.iban, ROUND(AVG(amount), 2) AS mitjana_amount FROM transaction INNER J...	371 row(s) returned	0.078 sec / 0.000 sec

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```

322 -- **** Exercici 1 ****
323 • SELECT * FROM transaction;
324
325 • SELECT DATE(timestamp), SUM(amount) as suma
326 FROM transaction
327 GROUP BY DATE(timestamp)
328 ORDER BY suma DESC
329 LIMIT 5;

```

DATE(timestamp)	suma
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

Result 65 x

#	Time	Action	Message	Duration / Fetch
135	12:01:11	SELECT DATE(timestamp), SUM(amount) as suma FROM transaction ORDER BY suma DESC LIM...	Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains no...	0.000 sec
136	12:03:07	SELECT DATE(timestamp), SUM(amount) as suma FROM transaction GROUP BY DATE(timestamp) ...	5 row(s) returned	0.141 sec / 0.000 sec

Em lla el fet que em demana la data de cada transacció.

El problema aquí és el format de la data. He fet servir DATE a la columna timestamp per poder treure la data en el format any-mes-dia.

A partir d'aquí he fet la consulta normal agrupant les *amount* per data i limitant a 5 entrades, els 5 dies amb més ingressos.

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```

412 -- **** Exercici 2 ****
413 • USE nova_transaction;
414 • SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount
415 FROM companies
416 INNER JOIN transaction ON companies.company_id = transaction.business_id
417 WHERE amount > 350 AND amount < 400 AND (
418     DATE(transaction.timestamp) = '2015-04-29'
419     OR DATE(transaction.timestamp) = '2018-07-20'
420     OR DATE(transaction.timestamp) = '2024-03-13')
421 ORDER BY amount DESC;
422

```

company_name	phone	country	fecha	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21

Result 23 x

#	Time	Action	Message	Duration / Fetch
60	11:42:30	SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount FROM c...	Error Code: 1054. Unknown column 'company.id' in 'on clause'	0.000 sec
61	11:43:43	SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount FROM c...	6 row(s) returned	0.078 sec / 0.000 sec

Faig un INNER JOIN per poder extreure la informació de les companyies i poder-ho ajuntar amb les transactions amb operadors consecutius.

Exercici 3

Informació sobre la quantitat de transaccions que realitzen les empreses. Cal especificar si tenen igual o més de 400 transaccions o menys.

Aquesta és la primera solució. El problema que té és que l'usuari ha de fer un treball extra per discriminar quines han fet més de 400 transaccions i quines menys:

```

358 • SELECT count(transaction.id), company_name
359 FROM transaction
360 INNER JOIN company ON transaction.company_id = company.id
361 GROUP BY transaction.company_id, company.company_name;
362

```

Result Grid

count(transaction.id)	company_name
2401	Ac Fermentum Incorporated
410	Magna A Neque Industries
447	Fusce Corp.
1514	Convallis In Incorporated
472	Ante Iaculis Nec Foundation
449	Donec Ltd
1541	Sed Nunc Ltd
1511	Amet Nulla Donec Corporation
474	Magnahe Dolorque Mor Inc

Output

#	Time	Action	Message	Duration / Fetch
181	13:46:45	SELECT count(id), company_name FROM transaction INNER JOIN company ON transaction.compan...	Error Code: 1052. Column 'id' in field list is ambiguous	0.000 sec
182	13:48:11	SELECT count(transaction.id), company_name FROM transaction INNER JOIN company ON transact...	100 row(s) returned	0.234 sec / 0.000 sec

A partir de la solució proposada, afegim un CASE per que escrigui una columna extra amb el detall del número de transaccions de la companyia. Funciona com un IF/ELSE. Quan el count és > o = de 400, escriu el text 'igual o més de 400'. Cas contrari serà 'menys de 400':

```

SELECT business_id, COUNT(transaction.id) AS numero_transaccions,
CASE WHEN COUNT(transaction.id) >= 400 THEN 'igual o més de 400'
ELSE 'menys de 400' END AS categoria, company_name
FROM transaction
INNER JOIN company ON transaction.company_id = company.id
GROUP BY business_id, company_name;

```

Result Grid

business_id	numero_transaccions	company_name	categoria
b-2362	380	Lorem Eu Incorporated	menys de 400
b-2286	397	Fringilla LLC	menys de 400
b-2470	399	Nec Luctus LLC	menys de 400
b-2578	402	Dui Quis Institute	igual o més de 400
b-2446	407	Risus Associates	igual o més de 400
b-2226	410	Magna A Neque Industries	igual o més de 400
b-2526	411	Amet Luctus Vulputate Foundation	igual o més de 400
b-2298	418	Elit Etiam Laoreet Associates	igual o més de 400
b-2298	418	Magnahe Dolorque Mor Inc	igual o més de 400

Output

#	Time	Action	Message	Duration / Fetch
24	19:24:06	SELECT business_id, COUNT(transaction.id) AS numero_transaccions, CASE WHEN COUNT(tran...	100 row(s) returned	0.187 sec / 0.000 sec
25	19:24:35	SELECT business_id, COUNT(transaction.id) AS numero_transaccions, company_name, CASE W...	100 row(s) returned	0.156 sec / 0.000 sec

Exercici 4

Borrar registro de la tabla ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
Primer verifiquen que el registre existeix:

```

468 -- **** Exercici 4 ****
469 # ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD
470 • SELECT ID FROM transaction
471 WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
472

```

Result Grid

ID
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD

transaction 15

Output

#	Time	Action	Message	Duration / Fetch
26	19.32.18	SELECT ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' FROM transaction	100000 row(s) returned	0.000 sec / 0.047 sec
27	19.32.48	SELECT ID FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.000 sec / 0.000 sec

Apareix a l'output. Ara procedim a borrar-lo amb DELETE FROM.

```

472
473 • DELETE FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
474

```

Output

#	Time	Action	Message	Duration / Fetch
27	19.32.48	SELECT ID FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.000 sec / 0.000 sec
28	19.34.17	DELETE FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.047 sec

El procés és OK. Hi ha un registre esborrat (1 row(s) affected).

Comprovem de nou que no hi sigui:

```

475 • SELECT ID FROM transaction -- refaig consulta per comprobar que no hi ha el registre
476 WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
477

```

Result Grid

ID

transaction 16

Output

#	Time	Action	Message	Duration / Fetch
28	19.34.17	DELETE FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.047 sec
29	19.35.48	SELECT ID FROM transaction -- refaig consulta per comprobar que no hi ha el registre WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec

L'output està buit.

Exercici 5

Crea una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

```

371 • CREATE VIEW VistaMarketing AS
372 SELECT company_name, phone, country, AVG(amount) AS mitjana
373 FROM company
374 INNER JOIN transaction ON transaction.company_id = company.id
375 GROUP BY company_name, phone, country
376 ORDER BY mitjana DESC;
377
378 • SELECT * FROM VistaMarketing;
379

```

company_name	phone	country	mitjana
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Pretium Neque Corp.	07 77 48 55 28	Australia	276.158330
Urnna Conwallis Associates	06 01 24 77 04	United States	274.235011
At Associates	09 56 61 10 65	New Zealand	272.214870
Metus Vitae Associates	08 25 44 40 66	Australia	270.080965
Aliquet Diam Limited	02 76 61 47 46	United States	269.599181
Nec Luctus LLC	02 14 71 75 73	Norway	268.604837
Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.850372
Tristique Mass Prosserda Company	01 15 07 77 16	United States	267.816186

VistaMarketing 95 x

Output

#	Time	Action	Message	Duration / Fetch
194	14:04:15	CREATE VIEW VistaMarketing AS SELECT company_name, phone, country, AVG(amount) AS mitjan...	0 row(s) affected	0.015 sec
195	14:04:29	SELECT * FROM VistaMarketing	100 row(s) returned	0.547 sec / 0.000 sec

Hem creat primer la vista com una consulta normal amb un inner join. Després invoco la vista amb un select normal i la taula és la vista.

Nivell 3

Exercici 1

Estat de les targetes. Quines estan actives?

Si les tres últimes transaccions han estat declinades aleshores és inactiu.

Creem una taula per recollir l'estat de les targetes:

```

411 -- Creo una tabla temporal con las últimas 3 transacciones por tarjeta
412 • CREATE TABLE card_status AS
413 WITH last_transactions AS (
414     SELECT
415         transaction.card_id,
416         transaction.declined,
417         ROW_NUMBER() OVER (PARTITION BY transaction.card_id ORDER BY transaction.timestamp DESC) AS row_num
418     FROM transaction
419 ),
420 calculated_status AS (
421     SELECT
422         credit_card.id,

```

Output

#	Time	Action	Message	Duration / Fetch
13	09:09:11	CREATE TABLE card_status AS WITH last_transactions AS (SELECT transaction.card_id, ...	Error Code: 1054. Unknown column 'credit_card.card_type' in field list'	0.031 sec
14	09:10:36	CREATE TABLE card_status AS WITH last_transactions AS (SELECT transaction.card_id, ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.922 sec

Anomenem la taula `card_status` i fem servir les funcions de FINESTRA de MySQL.

Dins d'aquesta taula creem una subconsulta que podem invocar més tard amb el nom de últimas. És una CTE (common table expression) que s'inicia amb un `WITH` i amb un alias, per poder-la invocar després (aquí anomenada `ORDEN`).

Aquesta CTE contindrà una FINESTRA de MySQL. Concepte nou per a mí. El `ROW_NUMBER` genera una numeració per cada tarjeta (el `PARTITION` que ve després de `OVER`).

S'ordena per data (aquesta s'ha de modificar per fer servir el format `DATE` de any-mes-dia).

A continuació hem de fer els filtres oportuns. Fem servir `CASE` al que anomenem *estado* que funciona com un `if/else` en altres llenguatges. En concret ens porta un contador de control per sumar les operacions que son *declined*.

Si les tres últimes son declined, aleshores el seu estat és *inactiva*. La resta son actives. A la consulta es pot veure que les 5000 targetes son totes actives.

```

436 -- Veo cuántas tarjetas están activas
437 • SELECT
438     status,
439     COUNT(*) AS total_cards
440 FROM card_status
441 GROUP BY status;
442

```

Result Grid

status	total_cards
ACTIVA	5000

Output

#	Time	Action	Message	Duration / Fetch
15	09:12:32	CREATE TABLE card_status AS WITH last_transactions AS (SELECT transaction.card_id, ...	Error Code: 1050. Table 'card_status' already exists	0.000 sec
16	09:14:05	SELECT status, COUNT(*) AS total_cards FROM card_status GROUP BY status	1 row(s) returned	0.015 sec / 0.000 sec

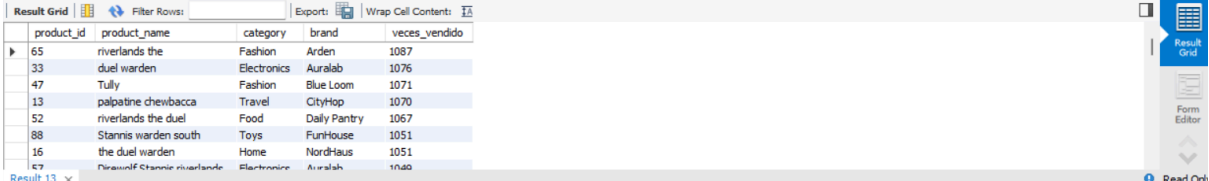
Exercici 2

Creo la taula products i l'omple amb les dades del CSV. Té 100 registres.

```

445 -- Creo la tabla products
446 CREATE TABLE products (
447     id INT PRIMARY KEY,
448     product_name VARCHAR(100),
449     price VARCHAR(20),
450     colour VARCHAR(20),
451     weight DECIMAL(5,2),
452     warehouse_id VARCHAR(10),
453     category VARCHAR(50),
454     brand VARCHAR(50),
455     cost VARCHAR(20),
456     launch_date DATE
457 );

```



Result 13 x

product_id	product_name	category	brand	veces_vendido
65	riverlands the	Fashion	Arden	1087
33	duel warden	Electronics	Auralab	1076
47	Tully	Fashion	Blue Loom	1071
13	palpatine chewbacca	Travel	CityHop	1070
52	riverlands the duel	Food	Daily Pantry	1067
88	Stannis warden south	Toys	FunHouse	1051
16	the duel warden	Home	NordHaus	1051
127	Dunsford Stannis riverlands	Electronics	Auralab	1040

Output

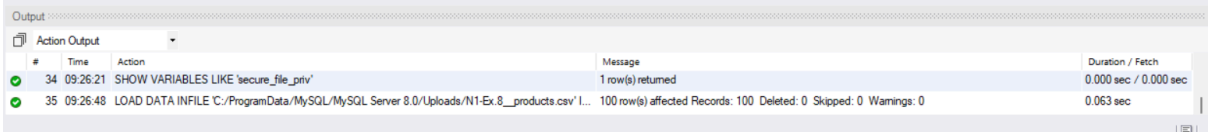
#	Time	Action	Message	Duration / Fetch
16	09:14:05	SELECT status, COUNT(*) AS total_cards FROM card_status GROUP BY status	1 row(s) returned	0.015 sec / 0.000 sec
17	09:18:30	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(100), price VA...	0 row(s) affected	0.062 sec

He obert el CSV de products i he vist que cost y price tenen el símbol \$. És per això que carrego aquestes columnes com a VARCHAR. Després les hauré de modificar per que sigui coherent. Carrego doncs els valors del csv que ens donen.

```

505 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'
506 INTO TABLE products
507 CHARACTER SET utf8mb4
508 FIELDS TERMINATED BY ','
509 ENCLOSED BY ''
510 LINES TERMINATED BY '\n'
511 IGNORE 1 ROWS
512 (id, product_name, price, colour, weight, warehouse_id, category, brand, cost, launch_date);
513 SHOW VARIABLES LIKE 'secure_file_priv';

```



Output

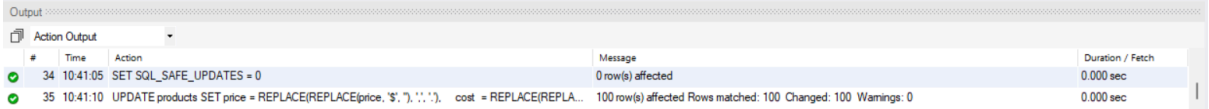
#	Time	Action	Message	Duration / Fetch
34	09:26:21	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.000 sec / 0.000 sec
35	09:26:48	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv' I...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.063 sec

Ara elimino el símbol \$ de cost i price:

```

542 SET SQL_SAFE_UPDATES = 0; -- eliminamos los simbolos $ de los valores de cost y price
543 UPDATE products
544 SET price = REPLACE(REPLACE(price, '$', ''), ',', '.'),
545     cost = REPLACE(REPLACE(cost, '$', ''), ',', '.');
546 SET SQL_SAFE_UPDATES = 1;
547

```



Output

#	Time	Action	Message	Duration / Fetch
34	10:41:05	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
35	10:41:10	UPDATE products SET price = REPLACE(REPLACE(price, '\$', ''), ',', '.'). cost = REPLACE(REPLA...	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0	0.000 sec

I per últim edito els camps VARCHAR de cost i price i els passo a DECIMAL:

```

548 • ALTER TABLE products MODIFY COLUMN price DECIMAL(10,2); -- cambio el tipo de dato de la columna para que sea coherente al contenido
549 • ALTER TABLE products MODIFY COLUMN cost DECIMAL(10,2);
550
Output
Action Output
# Time Action Message Duration / Fetch
37 10:41:35 ALTER TABLE products MODIFY COLUMN price DECIMAL(10,2) 100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0 0.047 sec
38 10:41:38 ALTER TABLE products MODIFY COLUMN cost DECIMAL(10,2) 100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0 0.062 sec

```

Ara les dades son totes coherents. Els camps numèrics no son tractats com a VARCHAR i es podrà operar amb ells.

La taula *transaction* té diversos productes a cada fila. No es pot unir *transaction* y *products* amb una *foreign key* normal. Cal manipular els valors primer.

La funció de mySQL FIND_IN_SET() busca un valor dins d'una llista de valors separats per comes i ens torna la seva posició per tots aquells que s'han venut al menys una vegada. Malgrat que he tingut MOLTES dificultats per arribar a aquesta solució, però no s'ajusta al que demana l'enunciat.

L'enunciat ens demana explícitament crear una taula que relacioni products amb transaction. Es prenen les dades de la columna product_ids com una llista.

Primer doncs creem la taula amb la id de la transacció i la id del producte amb les seves FOREIGN KEYS per relacionar *transaction* i *products*.

```

568 # ***** solucion propuesta después de corrección *****
569 -- Tabla intermedia para unir los productos con las transacciones
570 • CREATE TABLE transaction_products (
571     transaction_id VARCHAR(40) NOT NULL,
572     product_id INT NOT NULL,
573     PRIMARY KEY (transaction_id, product_id),
574     FOREIGN KEY (transaction_id) REFERENCES transaction(id),
575     FOREIGN KEY (product_id) REFERENCES products(id)
576 );

```

```

Output
Action Output
# Time Action Message Duration / Fetch
43 10:58:18 DROP TABLE 'nova_transaction', 'transaction_products' 0 row(s) affected 0.062 sec
44 10:58:31 CREATE TABLE transaction_products (transaction_id VARCHAR(40) NOT NULL, product_id INT... 0 row(s) affected 0.047 sec

```

A continuació inserim una nova columna desempaquetant cada id del producte venut i relacionant-lo amb la id de la transacció:

```

578 -- JSON_TABLE (JavaScript Object Notation) separa la lista en filas
579 • INSERT INTO transaction_products (transaction_id, product_id)
580 SELECT transaction_id, jt.product_id
581 FROM transaction
582 JOIN JSON_TABLE(
583   CONCAT('[', transaction.product_ids, ']'),
584   '$[*]' COLUMNS (product_id INT PATH '$'))
585 ) AS jt;

```

transaction_id	product_id
001A60EA-DC9C-4E5A-9460-6628B100E7E1	1
0032F08B-8BE6-4AA5-85EE-EEAD533C0C48	1
00342381-503D-422D-85AB-F2D4FFAAD4C7	1
004C0A80-E537-46D8-BE44-343D2176DF15	1
004D1DB5-82CB-4460-98B6-31C42CA96E5F	1
0062599C-0A1F-4405-AD55-45E20043B551	1

#	Time	Action	Message	Duration / Fetch
44	10:58:31	CREATE TABLE transaction_products (transaction_id VARCHAR(40) NOT NULL, product_id INT...	0 row(s) affected	0.047 sec
45	11:02:28	INSERT INTO transaction_products (transaction_id, product_id) SELECT transaction_id, jt.product_id ...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	8.219 sec

Comprovem que l'output sigui el que buscàvem:

```

586
587 • select * from transaction_products;
588

```

transaction_id	product_id
001A60EA-DC9C-4E5A-9460-6628B100E7E1	1
0032F08B-8BE6-4AA5-85EE-EEAD533C0C48	1
00342381-503D-422D-85AB-F2D4FFAAD4C7	1
004C0A80-E537-46D8-BE44-343D2176DF15	1
004D1DB5-82CB-4460-98B6-31C42CA96E5F	1
0062599C-0A1F-4405-AD55-45E20043B551	1

#	Time	Action	Message	Duration / Fetch
45	11:02:28	INSERT INTO transaction_products (transaction_id, product_id) SELECT transaction_id, jt.product_id ...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	8.219 sec
46	11:03:42	select * from transaction_products	253388 row(s) returned	0.000 sec / 0.157 sec

I veiem que un product_id té varies transaction_id's.

Ara fem la consulta per comptar el número de vegades que s'ha venut cada producte. Hem de fer un INNER JOIN amb products si volem veure el nom del producte també:

```

589 • SELECT product_id, products.product_name, count(product_id) AS veces_vendido FROM transaction_products
590 INNER JOIN products ON transaction_products.product_id = products.id
591 GROUP BY product_id
592 ORDER BY veces_vendido DESC;

```

product_id	product_name	veces_vendido
52	riverlands the duel	2654
29	Tully maester Tully	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2600
87	sith Jade	2597

#	Time	Action	Message	Duration / Fetch
52	11:09:47	SELECT product_id, count(product_id) AS veces_vendido FROM transaction_products GROUP BY ...	100 row(s) returned	0.078 sec / 0.000 sec
53	11:12:57	SELECT product_id, products.product_name, count(product_id) AS veces_vendido FROM transactio...	100 row(s) returned	0.265 sec / 0.000 sec

Al output les dades estan classificades en ordre descendent.